

SQL

Lenguaje de
consultas
Relacionales

SQL no es un lenguaje de programación

Fue diseñado con el único propósito de **acceder a datos estructurales**

Como en el caso de los más modernos lenguajes relacionales, SQL está basado en el **cálculo relacional de tuplas**.

Toda consulta formulada utilizando el **cálculo relacional** de tuplas (o su equivalente, el **álgebra relacional**) se puede formular también utilizando SQL.

Otras características de SQL

Capacidades que van más allá del cálculo o del álgebra relacional. Características EXTRAS proporcionadas por SQL:

Capacidades aritméticas

- En SQL es posible incluir operaciones aritméticas así como comparaciones, por ejemplo $A < B + 3$.

Funciones agregadas

- Operaciones tales como *promedio* (*average*), *suma* (*sum*), *máximo* (*max*), etc., se pueden aplicar a las columnas de una relación para obtener una cantidad única.

Sentencia SELECT

SELECT [ALL | DISTINCT]

{ * | expr_1 [AS c_alias_1] [, ... [, expr_k [AS c_alias_k]] }

FROM table_name_1 [t_alias_1] [, ... [, table_name_n
[t_alias_n]]]

[**WHERE** condition]

[**GROUP BY** name_of_attr_i [, ... [, name_of_attr_j]]

[**HAVING** condition]]

[{**UNION** [ALL] | **INTERSECT** | **EXCEPT**} SELECT ...]

[**ORDER BY** name_of_attr_i [ASC | DESC] [, ... [,
name_of_attr_j [ASC | DESC]]];

Cláusulas comando SELECT (2)

GROUP BY

- La cláusula **GROUP BY** permite **agrupar filas según las columnas que se indiquen como parámetros**,
- Permite funciones de agregación para **obtener datos resumidos y agrupados** por las columnas que se necesiten

HAVING

- Permite un nuevo filtrado, **pero sobre las tuplas afectadas por el GROUP BY**, en función de una condición aplicable a cada grupo de filas

Ejemplos con GROUP BY

Si queremos conocer la lista de pedidos por cliente, utilizaremos la siguiente consulta:

```
SELECT cliente, fechapedido, monto  
FROM Clientes inner join Pedidos  
using(idcliente) ORDER BY cliente;
```

Sin embargo, lo primero que uno supone es que se utiliza Group By para “agrupar” las tuplas del cliente.

Con la cláusula **GROUP BY** no funciona

Ejemplos con GROUP BY ₂

Si queremos conocer la CANTIDAD de pedidos por cliente, utilizaremos la siguiente consulta:

```
SELECT idcliente, cliente,  
count(fechapedido) as cant_pedidos  
FROM Clientes inner join Pedidos  
USING(idcliente)  
GROUP BY idcliente;
```

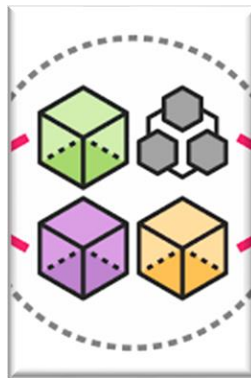
Como se observa, aparece una **función de Agregación**

Agregación por grupos



SQL permite separar tuplas de una tabla en grupos.

En estas condiciones, los operadores agregados ya descriptos pueden aplicarse a los **grupos**.



Esto quiere decir que: el valor del operador agregado no se calculan sobre todos los valores de la columna especificada, **sino sobre todos los valores de un grupo**.

Agregación por grupos



El operador agregado se **calcula** individualmente **para cada grupo**.

El agrupamiento de las tuplas en grupos se hace utilizando las palabras clave **GROUP BY** seguidas de una lista de atributos que definen los grupos.

En una consulta utilizando GROUP BY **con operadores agregados**, para dar sentido a los atributos agrupados, **debemos primero obtener la lista objetivo**

Funciones de agregación

Las funciones de agregación en SQL nos permiten efectuar **operaciones sobre un conjunto de resultados**, pero devolviendo un **único valor** agregado para todos ellos.

COUNT

- Devuelve el número total de filas o tuplas seleccionadas por la consulta.

MIN

- Devuelve el valor mínimo del atributo que especifiquemos

MAX

- Devuelve el valor máximo del atributo que especifiquemos

SUM

- Suma los valores del atributo que especifiquemos.
- Sólo se puede utilizar en columnas numéricas

AVG

- devuelve el valor promedio del campo que especifiquemos. Sólo atributos numéricos

Funciones agregadas

Si queremos conocer el PRECIO promedio de todos los productos de la tabla PRODUCTOS, utilizaremos la siguiente consulta:

```
SELECT AVG(PRECIO) AS PROM_PRECIO  
FROM PRODUCTOS;
```

Si queremos conocer con cuántos Clientes contamos en la tabla CLIENTES, utilizaremos la instrucción:

```
SELECT COUNT(IDCLIENTE)  
FROM CLIENTES;
```

Funciones agregadas 2

Si queremos conocer el PRECIO MÁS ALTO de todos los productos de la tabla PRODUCTOS, utilizaremos la siguiente consulta:

```
SELECT MAX(PRECIO) AS MAS_CARO  
FROM PRODUCTOS;
```

Si queremos conocer cuántos Pedidos se hicieron durante el año 2015, utilizaremos la instrucción:

```
SELECT COUNT(*) FROM PEDIDOS  
WHERE YEAR(FECHAPEDIDO)=2015;
```

Funciones agregadas 2

Observemos la siguiente consulta sobre la tabla PEDIDOS:

```
SELECT COUNT(*) AS Cant_Pedidos,  
MIN(FechaPedido) AS Desde,  
MAX(FechaPedido) AS Hasta,  
SUM(Monto) AS MontoTotal,  
AVG(Monto) AS MontoPromedio  
FROM PEDIDOS
```

Como se puede observar del resultado de la consulta anterior, **las funciones de agregación devuelven una sola fila**, salvo que vayan unidas a la cláusula GROUP BY, que veremos a continuación

Agregación por grupos- ejemplo

- *Averiguar el sueldo promedio por cargo.
(se usa agrupamiento)*

```
select cargo, avg(sueldo) as SueldoPromedio  
from Empleados inner join Cargos using(idcargo)  
group by cargo
```

Agregación por grupos- ejemplo 2

- *Averiguar cantidad de familiares por empleado.*

```
SELECT empleado, count(idfliar) AS Cantidad  
from Empleados inner join Familiares  
using(idempleado)  
group by empleado
```

obtenemos varios grupos
y ahora podemos aplicar
el operador agregado
COUNT para cada grupo,
obteniendo el resultado
total de la consulta

Importante

- La cláusula GROUP BY se puede utilizar con más de un campo al mismo tiempo. Si indicamos más de un campo como parámetro nos devolverá la información agrupada por los registros que tengan el mismo valor en los campos indicados.

➤ **Nota:** Es muy importante tener en cuenta que cuando utilizamos la cláusula GROUP BY, los únicos campos que podemos incluir en el SELECT sin que estén dentro de una función de agregación, son los que vayan especificados en el GROUP BY.

Cláusula HAVING

La cláusula HAVING trabaja de forma muy parecida a la cláusula WHERE, y se utiliza para considerar sólo aquellos grupos que satisfagan la cualificación dada en la misma.

Las expresiones permitidas en la cláusula HAVING deben involucrar funciones agregadas.

Por otro lado, toda expresión que involucre funciones agregadas debe aparecer en la cláusula HAVING

Cláusula HAVING - ejemplo

- *Averiguar los cargos cuyo sueldo promedio es mayor a \$ 30000*

```
SELECT cargo, avg(sueldo) as SueldoPromedio  
FROM Empleados inner join Cargos using(idcargo)  
GROUP BY cargo  
HAVING SueldoPromedio >= 30000
```

Cláusula HAVING – ejemplo 2

- *Listado de los proveedores que nos venden menos de 3 artículos, utilizaremos la consulta:*

```
SELECT proveedor, count(idproducto) as cantidad
FROM Proveedores inner join Productos
using(idproveedor)
GROUP BY proveedor
HAVING cantidad<3
```



Funciones en MySQL

¿Por/para qué usar funciones?

Funciones en MySQL

- MySQL puede hacer mucho más que simplemente almacenar y recuperar datos .

- También podemos ***realizar manipulaciones en los datos*** antes de recuperarlos o guardarlos. Ahí es donde entran las Funciones de MySQL.

- Las funciones son simplemente ***piezas de código*** que realizan algunas operaciones y luego ***devuelven un resultado***.

Tipos de Funciones en MySQL

► Funciones integradas

Las funciones integradas son simplemente funciones que ya vienen *implementadas* en el servidor MySQL.

Estas funciones nos permiten realizar diferentes tipos de manipulaciones en los datos.

Las funciones integradas se pueden clasificar básicamente en las siguientes categorías más utilizadas: *numéricas, de cadena, de fecha, etc*

Ejemplos de SELECT con funciones

```
SELECT empleado, sueldo,  
round(sueldo*0.13,2) as Aporte_Jub  
FROM Empleados
```

```
SELECT concat(direccion,' - ',ciudad) as  
domicilio_completo FROM Proveedores;
```

```
SELECT empleado,  
timestampdiff(year, fechanacimiento, curdate())  
FROM Empleados
```

Más Ejemplos

Funciones de Cadena

Una función de cadena realiza operaciones en datos de texto. Utiliza una función de cadena para: limpiar datos, convertir datos a un formato diferente, cambiar mayúsculas y minúsculas, calcular métricas sobre los datos o realizar otras manipulaciones

EJ.: Mostrar todos los nombres de Proveedores con Mayúsculas:

```
SELECT UPPER(proveedor) FROM Proveedores
```

EJ.: Mostrar todos los productos con su detalle completo.

```
SELECT CONCAT(producto," en ", descripción)  
FROM Productos
```


Más Ejemplos

Funciones numéricas

Mostrar el IVA de los montos de la tabla Pedidos, lo que significa el 21% del mismo

De acuerdo a cómo se implementa la operación dependerá el tipo de resultado

- `SELECT monto*21/100 FROM Pedidos`
(devuelve 6 decimales)
- `SELECT monto*0.21 FROM Pedidos` (devuelve 4 decimales)
- `SELECT round(monto*0.21, 2) FROM Pedidos`
(para dejar a 2 decimales)

Más Ejemplos

Funciones de Fechas

- *Mostrar las fechas de pedido sin mostrar el dato hora*

```
SELECT DATE(fechapedido) FROM Pedidos  
ORDER BY fechapedido
```

- *Mostrar la diferencia en días entre la fecha de entrega y la fecha de envío;*

```
SELECT DATEDIFF(FECHAENTREGA,FECHAENVIO)  
FROM PEDIDOS
```

Más Ejemplos

Funciones de Fechas

- *Mostrar la fecha y el monto de cada pedido, de los últimos 5 pedidos*

```
SELECT date(fechapedido),monto  
FROM Pedidos  
ORDER BY fechapedido DESC  
LIMIT 5
```

Más Ejemplos

Funciones de Fechas

- *Listar todos los empleados y su edad, a partir de la fecha de nacimiento.*

```
SELECT empleado,  
timestampdiff(year, fechanacimiento,  
curdate())  
FROM Empleados
```

*¿Cómo averiguamos
cuanto tiempo lleva
en la empresa?*

Más Ejemplos

Mostrar campos numéricos como texto

- `SELECT monto from Pedidos`
- `SELECT concat('$ ', cast(monto as char))
from Pedidos`
- `SELECT concat(cast(monto as char), '
pesos') from Pedidos`



Condiciones WHERE



En una cláusula WHERE Se pueden combinar **múltiples condiciones** (de cualquiera de los tipos) mediante:

Los operadores lógicos **AND, OR y NOT**, determinando con el uso de **paréntesis** el orden de evaluación

(y contrarrestando el efecto de la prioridad de operadores).

Dónde buscar

- *Funciones para cadenas de caracteres*

<http://download.nust.na/pub6/mysql/doc/refman/5.0/es/string-functions.html>

- *Funciones y operadores numéricos*

<https://dev.mysql.com/doc/refman/8.0/en/numeric-functions.html>

- *Funciones Matemáticas*

<http://download.nust.na/pub6/mysql/doc/refman/5.0/es/mathematical-functions.html>

- *Funciones de Fecha y Hora*

<http://download.nust.na/pub6/mysql/doc/refman/5.0/es/date-and-time-functions.html>



¡¡Muchas Gracias !!

Consultas en el foro

